

PART 2: Node.js

Theory Questions

1. What is Node.js?

Answer: Node.js is a JavaScript runtime built on Google's V8 engine that allows JavaScript to run outside the browser for building server-side applications.

2. Why do we use Node.js?

Answer: We use Node.js because it is fast, scalable, lightweight, and supports asynchronous programming. It is ideal for building REST APIs and real-time applications.

3. Is Node.js a programming language?

Answer: No. Node.js is a JavaScript runtime environment. JavaScript is the programming language.

4. What is the V8 Engine?

Answer: V8 is Google's JavaScript engine that compiles JavaScript into machine code, making execution faster.

5. Why is Node.js fast?

Answer: Node.js is fast because it uses the V8 engine and follows a non-blocking, event-driven architecture.

6. Is Node.js single-threaded?

Answer: Yes. Node.js uses a single main thread with an Event Loop to handle multiple requests efficiently.

7. What is the Event Loop?

Answer: The Event Loop manages asynchronous operations and executes callbacks when tasks are completed.

8. What is Blocking I/O?

Answer: Blocking I/O waits for one operation to finish before starting the next operation.

9. What is Non-Blocking I/O?

Answer: Non-blocking I/O starts an operation and immediately continues with other tasks without waiting.

10. Difference between Blocking and Non-Blocking I/O?

Answer:

Blocking	Non-Blocking
----------	--------------

Waits	Doesn't wait
-------	--------------

Slower	Faster
--------	--------

One task	Multiple tasks
----------	----------------

11. What is Synchronous Programming?

Answer: Synchronous programming executes tasks one after another and waits for each task to finish.

12. What is Asynchronous Programming?

Answer: Asynchronous programming allows multiple tasks to run without waiting for previous tasks to complete.

13. Difference between Synchronous and Asynchronous?

Synchronous	Asynchronous
Waits	Doesn't wait
Blocking	Non-blocking
One by one	Multiple tasks

14. What is a Callback?

Answer: A callback is a function passed to another function that runs after an operation completes.

15. What is Callback Hell?

Answer: Callback Hell is a situation where many nested callbacks make the code difficult to read and maintain.

16. What is a Promise?

Answer: A Promise is an object that represents the eventual success or failure of an asynchronous operation.

17. What are the states of a Promise?

Answer:

- Pending
- Fulfilled
- Rejected

18. What is `async/await`?

Answer: `async/await` is a cleaner way to write asynchronous code using Promises.

19. What is `__dirname`?

Answer: `__dirname` returns the absolute path of the current directory.

20. What is `__filename`?

Answer: `__filename` returns the absolute path of the currently executing file.

Practical Questions

21. Why did you choose Node.js for your project?

Answer: I chose Node.js because it is fast, lightweight, and allows me to use JavaScript for both frontend and backend. It also makes it easy to build REST APIs and handle many users efficiently.

22. Where did you use Node.js in your project?

Answer: I used Node.js to build the backend server. It receives requests from the frontend, processes business logic, connects to the database, and returns JSON responses.

23. Where did you use `async/await`?

Answer: I used `async/await` for database operations such as creating, reading, updating, and deleting data. It made the code cleaner and easier to maintain.

24. Why did you use `async/await` instead of callbacks?

Answer: `async/await` improves code readability and works well with `try...catch` for error handling. It also avoids callback hell.

25. How does Node.js handle multiple users?

Answer: Node.js uses asynchronous programming and the Event Loop. While one request waits for a database or file operation, the server continues processing other requests.

26. How did you connect Node.js with the database?

Answer: I used Mongoose to connect Node.js with MongoDB and Sequelize to connect Node.js with MySQL. The connection string was stored in the `.env` file.

27. How did you handle errors?

Answer: I used `try...catch` blocks in asynchronous functions and returned proper HTTP status codes and error messages.

28. How did you test your APIs?

Answer: I tested my APIs using Postman by checking request methods, status codes, JSON responses, and error handling.

29. Why did you use environment variables?

Answer: I stored sensitive information such as the database URL, JWT secret, and API keys in a `.env` file to improve security and make deployment easier.

30. Explain the request flow in your Node.js project.

Answer: The client sends a request to the Express server. The request goes to the route, then the controller, which performs the required logic and interacts with the database if needed. Finally, the server sends a JSON response back to the client.

Scenario-Based Questions

31. What happens if 1,000 users access your API at the same time?

Answer: Node.js handles requests asynchronously using the Event Loop. It doesn't block while waiting for I/O operations, allowing it to serve many users efficiently.

32. What happens if the Event Loop is blocked?

Answer: If the Event Loop is blocked by a long-running task, the server cannot process new requests until that task finishes, causing slow responses.

33. Your API is slow. How would you debug it?

Answer: I would check server logs, identify slow database queries, optimize them, monitor CPU and memory usage, and reduce unnecessary processing.

34. What happens if the database connection fails?

Answer: The application should catch the error, log it, return an appropriate error response (such as HTTP 500), and avoid crashing the server.

35. Why should we use asynchronous code instead of synchronous code in Node.js?

Answer: Asynchronous code allows the server to handle multiple requests without waiting for one operation to finish, improving performance and scalability.

Top 10 Node.js Questions (Must Prepare)

- 1 What is Node.js?
- 2 Why do we use Node.js?
- 3 Why is Node.js fast?
- 4 What is the V8 Engine?
- 5 Is Node.js single-threaded?
- 6 What is the Event Loop?
- 7 Difference between synchronous and asynchronous programming.
- 8 What is a Promise?
- 9 What is async/await?
- 10 Explain the request flow in your Node.js project.